VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



Report

Topic: **Virtual Assistant**
Project: **Student Learning Assistant**

Course: Computational Thinking (CSC10014)

Team members
25127246 Võ Minh Triết
25127510 Nguyễn Phước Thọ
25127260 Nguyễn Dương An
25127087 Lư Gia Linh
25127201 Huỳnh Anh Khoa
24127449 Nguyễn Anh Minh

Instructors
Châu Thành Đức
Đỗ Đức Hào
Trần Hoàng Quân

Ho Chi Minh City, October 2026

Contents

Chapter 1. Introduction 1

1.1. Problem statement 1

1.2. Motivation 1

1.3. Project objectives 1

1.4. Target users 2

1.5. Project scope 2

1.6. Related approaches 2

Chapter 2. System requirements analysis 3

2.1. Overview and assumptions 3

2.2. Functional requirements 3

2.3. Non-functional requirements 4

2.4. Use case model 4

2.5. Use case specification: plan study tasks 5

2.6. Conceptual data model 6

2.7. Computational thinking in the requirements 8

References 10

Chapter 1. Introduction

1.1. Problem statement

University students manage assignments with deadlines, long-term study goals, course materials, and review sessions at the same time. Details about this work are often spread across notes, files, calendars, and messages. When a student starts with an informal request, they still need to decide what the tasks are, when each task is due, what to do first, and which materials are relevant. That effort grows when time is limited or plans change.

Planning, note keeping, and studying also tend to happen in separate places. A student may save useful notes and fail to find them when needed. After a study session, they may not turn what they learned into a clear next step. Research on self-regulated learning describes goal setting, progress monitoring, and strategy adjustment as parts of the learning process [2]. This project proposes a personal assistant that connects planning and task execution, knowledge management, and study support.

The project asks how a student's requests, learning materials, and progress can be represented as clear structures that the system can process and link. Those structures should help the student choose a next action while making missing information and unconfirmed assumptions visible. The assistant provides suggestions for the student to review.

1.2. Motivation

The project gives the team a practical way to apply computational thinking to a familiar problem. A broad study goal can be broken into smaller tasks. Repeated workflows can be identified, such as setting a goal, taking action, checking progress, and adjusting the plan. The system can represent shared concepts such as goals, tasks, materials, and study states, then apply explicit rules to them. Following Wing, the team treats computational thinking as a general way to organize and solve problems [1].

The three proposed features support different parts of a student's work. Planning turns an intention into actionable tasks. Knowledge management keeps study materials organized and retrievable. Study support gives the student a way to practise, receive feedback, and decide what to review next. A result from a study session can become a task, and saved material can be used in a later session. This scope reflects the team's analysis of decomposition, pattern recognition, and abstraction.

1.3. Project objectives

Overall objective

Design and build a prototype that helps students organize academic work, manage learning materials, and carry out review activities through clear data structures, rules, and workflows.

Specific objectives

1. Represent study requests as goals and tasks with deadlines, estimated durations, priorities, statuses, and relevant constraints. Use this information to propose a schedule and track progress.
2. Organize and retrieve notes, documents, and links by topic, and connect them to a goal or task when the user provides enough information.
3. Define a study workflow that uses an explanation or selected material, practice questions, feedback, and a suggested review step.

4. Define a shared context that lets the three features exchange information while keeping their responsibilities distinct.
5. Evaluate the prototype with synthetic test cases, including missing details, conflicting requests, searches with no results, and schedules that cannot meet the stated constraints.

1.4. Target users

The primary users are university students who need to coordinate coursework, study materials, and review activities. They can enter a broad goal or an informal request, answer follow-up questions, inspect and edit the suggested result, and choose what to do next. Instructors and team members may evaluate the prototype, but they are not its primary users.

1.5. Project scope

Proposed features

1. **Planning and task execution.** Accept a goal or request, break it into tasks, record deadlines, durations, priorities, dependencies, and availability when provided, then propose a schedule. The system identifies missing details and conflicts, and lets the student revise the plan and track task status.
2. **Knowledge management.** Save user-provided notes, documents, and links; organize them by topic or tag; and search for material related to a task or goal. Responses based on saved content must use the available source information. The system must not invent citations or present unverified claims as facts.
3. **Study support.** Support a study session around a topic or supplied material with an explanation, practice questions, feedback, and a suggestion for what to review. The student can add the suggested review activity to the plan.

The team will focus on workflows that can be demonstrated and tested within the course. Advanced integrations are outside the initial scope.

Out of scope

The project does not include synchronization with a real personal calendar, notifications through external services, storage of sensitive personal profiles, professional advice, guaranteed optimal schedules, or a complete adaptive learning platform. The prototype does not depend on a large language model or a paid service. Any automatically generated content added later must be checked and presented as a suggestion. Development and demonstration will use synthetic data instead of students' private calendars, emails, or task lists.

1.6. Related approaches

Calendar and task tools help users record work and due dates. Note-taking tools organize and search stored material. Learning tools provide course content or practice activities. This project studies how those activities could share a learning context, linking goals and tasks with materials and study sessions. The prototype will be evaluated to see whether those links help students identify a clear next action.

The project also draws on research about learning techniques. Dunlosky and colleagues review retrieval practice and distributed practice as techniques with research support [3]. The proposed study workflow uses questions, feedback, and a next review step. The team will not claim that the prototype improves learning outcomes without an evaluation that measures them.

Chapter 2. System requirements analysis

2.1. Overview and assumptions

The proposed system lets students use three features within a shared learning context. Students provide information, review results, answer clarification questions, edit suggestions, and decide whether to use them. The system processes user-provided or synthetic data. It does not access personal calendars, email, or accounts.

This chapter specifies the intended behaviour of the product. Inputs may be incomplete or inconsistent. In those cases, the system should ask for clarification, state its assumptions, or explain what it cannot process. It should not silently fill in missing details.

2.2. Functional requirements

ID	Proposed requirement
FR-01	The system lets students enter study goals and tasks, deadlines, estimated durations, priorities, dependencies, and available time through a form or a natural-language request.
FR-02	The system breaks a goal into trackable tasks. Each task has at least an identifier, title, and status; it may also have a deadline, duration, priority, and dependencies. User-provided facts are distinguished from system suggestions.
FR-03	The system detects required information that is missing or inconsistent and asks a focused clarification question. If the available details are still insufficient, it explains the limitation instead of creating a schedule that appears complete.
FR-04	The system proposes task priorities and a schedule based on deadlines, user-provided priorities or preferences, dependencies, and available time. The student reviews the result as a proposal, and the system does not guarantee an optimal schedule.
FR-05	The system identifies conflicts, insufficient time, and unscheduled tasks. It explains the relevant constraints and lets the student revise the details and request an updated plan.
FR-06	The system lets students review and update task status. When a status, deadline, or availability changes, they can request an updated plan.
FR-07	The system lets students save notes, links, and text they provide, and organize those items by title, topic, or tag.
FR-08	The system searches saved items by keyword or topic, shows matching content with its supplied source, and lets students link an item to a related goal or task. It does not invent a source when none is available.
FR-09	The system lets students start a study session from a topic or saved item, review an explanation or summary based on the available input, answer practice questions, and receive feedback.
FR-10	The system records basic study-session information, including the topic, progress, and material to review. It suggests a follow-up activity that the student can add to the plan as a task.
FR-11	The system uses a shared context to connect goals, tasks, knowledge items, and study sessions. Within the prototype, students can review, edit, or delete information they entered.

Why these three features

Planning turns an intention into work with an order and a time. Deadlines, durations, and availability help make the resulting plan useful for decisions. Knowledge management makes saved

material easier to find and connect to current work. Study support gives students a way to practise, check their understanding, and choose what to review. Research reviews identify retrieval practice as a learning technique with supporting evidence [3]. Each feature has its own input, process, and output, while the shared context lets them pass relevant information to one another.

2.3. Non-functional requirements

1. **Correctness and transparency:** show the supplied facts, missing details, assumptions, and warnings. Label schedules as proposals. Do not present an inferred priority as a confirmed user preference.
2. **Usability:** make the main results easy to scan. Ask only for information needed to continue the current workflow. Let students edit details without entering the entire request again.
3. **Privacy:** use synthetic data in examples and tests. Do not store credentials, personal calendars, email, or sensitive identifying information in the repository. Avoid logging user content unless processing requires it.
4. **Reliability:** validate input and provide actionable messages for invalid data or retrieval errors. An error should not discard content already entered in the current session.
5. **Modularity and testability:** define clear data boundaries between the three features. Test task breakdown, scheduling, retrieval, and study feedback independently with input and expected-output examples.
6. **Controlled extensibility:** allow for persistent storage or a language service to be added later, while keeping the course prototype independent of external services and paid models.

2.4. Use case model

The primary actor is the **Student**. Figure 2.1 shows the use cases the student interacts with, grouped by the three features. Optional steps, such as answering a clarification question or resolving a schedule conflict, appear as «extend» relations.

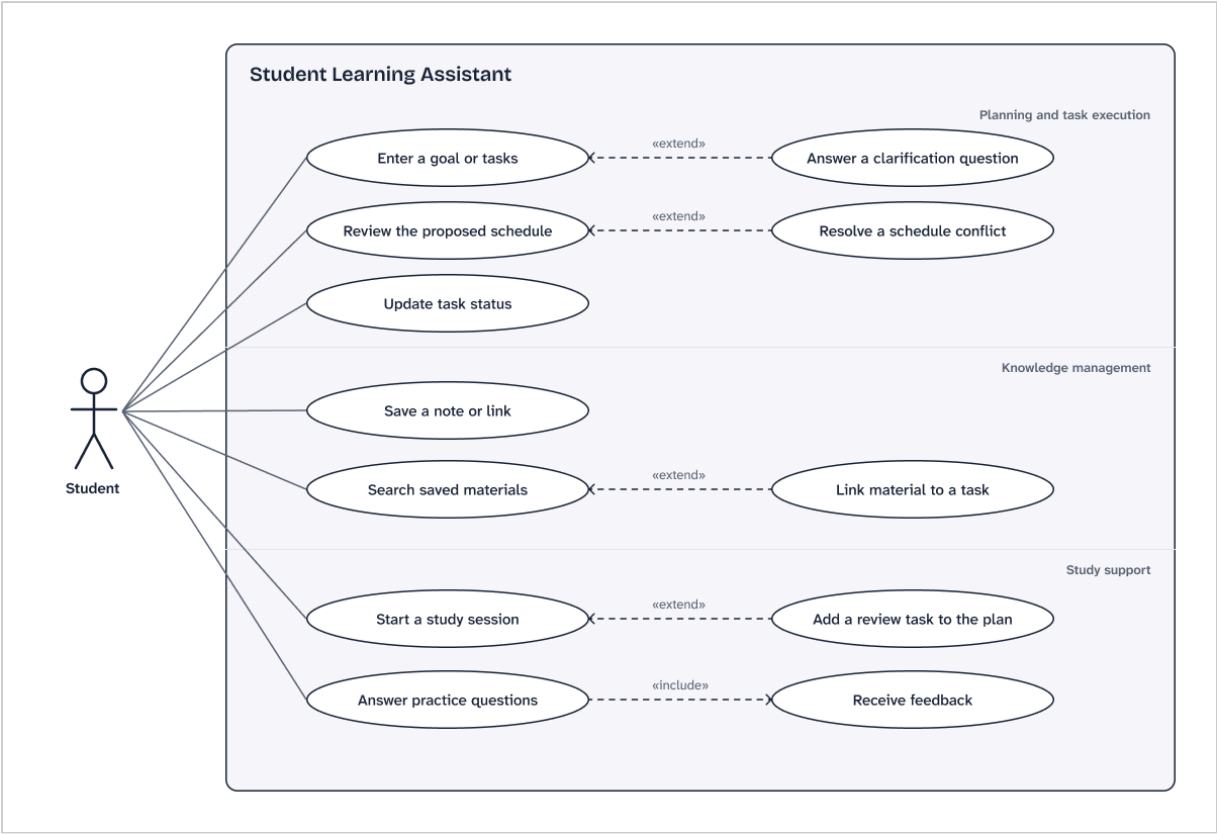


Figure 2.1. Use case diagram, grouped by feature.

2.5. Use case specification: plan study tasks

ID and name	UC-01, Plan and track study tasks
Primary actor	Student
Goal	Turn a goal or task list into structured tasks and a schedule proposal that the student can review and edit.
Preconditions	The student has a goal or at least one task to plan, and the system can accept input. A personal calendar connection is not required.
Trigger	The student opens the planning feature and enters a goal, tasks, or a request to change the current plan.
Postconditions	The system displays structured tasks, a proposed schedule when sufficient information is available, assumptions or warnings, and task statuses. The student can accept, edit, or recalculate the proposal.

Main flow

1. The student enters a goal or describes the work. They may include deadlines, durations, priorities, dependencies, and available time.
2. The system extracts identifiable facts, breaks the goal into tasks, and marks details that are missing.
3. The system presents a draft task list for review and distinguishes extracted facts from suggestions.
4. If the required information is available, the system orders tasks by dependencies and priority, then checks durations against availability and deadlines.

5. The system produces a proposed schedule, lists conflicts or unscheduled tasks, and explains the relevant constraints.
6. The student reviews the result, edits details, or updates task status.
7. When something changes, the system updates the plan and presents the revised proposal for the student to review.

Alternative and exception flows

1. **A1, Missing information:** If a detail needed for scheduling is missing, such as duration or availability, the system asks a focused question. After the student responds, the flow resumes at step 2. If they skip the question, the system can keep the task list and explain why it cannot schedule the work yet.
2. **A2, Conflict or infeasible schedule:** The system identifies the conflict and lists work it could not schedule. It asks the student to revise the constraints instead of presenting the result as feasible.
3. **A3, Student edits the result:** If the student changes a priority, deadline, or status, the system preserves unchanged facts, updates the affected details, and recalculates the proposal.
4. **A4, Unsupported request:** The system explains which part it does not support and does not invent data to complete the request.

Figure 2.2 shows a proposed chat screen for this use case. The assistant highlights the facts it extracted from the student's request and asks for the missing study hours (A1). The draft schedule then reports that the midterm review does not fit before the deadline (A2). The plan panel separates facts the student supplied from estimates the system assumed.

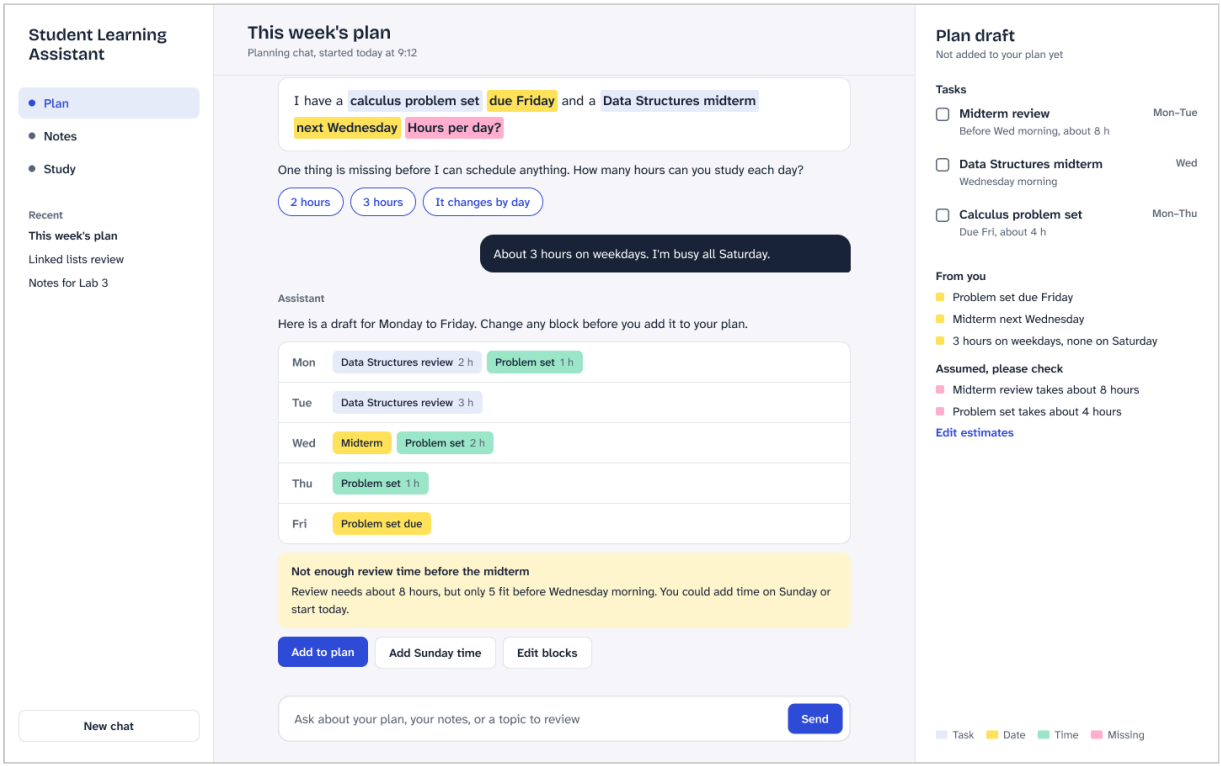


Figure 2.2. Proposed chat screen for planning study tasks.

2.6. Conceptual data model

The team proposes an *AssistantContext* for each learning session or context. It references the entities below so the three features can share information while keeping separate responsibilities. The

context does not decide what a student should value or prioritize. Each entity below maps to one table in the proposed database.

Entity	Proposed core attributes	Purpose and relationships
AssistantContext	context_id, user_goal, next_actions, created_at	Shared context linking goals with tasks, knowledge items, and study sessions.
Task	task_id, context_id, title, deadline?, duration_min?, priority?, status, source	A unit of work. The source field records whether the student supplied it or the system suggested it.
ScheduleBlock	block_id, task_id, start_at, end_at, status, assumption?	A proposed time slot for a task. Some tasks may remain unscheduled.
TaskDependency	task_id, depends_on_id	Records that one task cannot start before another task is finished.
KnowledgeItem	item_id, context_id, title, content_ref, source_url?, tags, created_at	User-provided material linked to a goal or task.
TaskKnowledge	task_id, item_id	Links a task to the materials it uses.
StudySession	session_id, context_id, topic, progress, review_notes, started_at	A learning session whose result can create a next action or review task.
StudySessionItem	session_id, item_id	Links a study session to the materials it covers.

A question mark marks an attribute that may be unknown. During implementation, the team will need to decide how to represent dates and time zones, validate input, and handle edits or deletions to linked records. The proposal does not select a database technology. Figure 2.3 shows these tables with their primary and foreign keys. Task dependencies and the links between materials and tasks or study sessions are many-to-many relationships, so each is stored in its own linking table: TaskDependency, TaskKnowledge, and StudySessionItem.

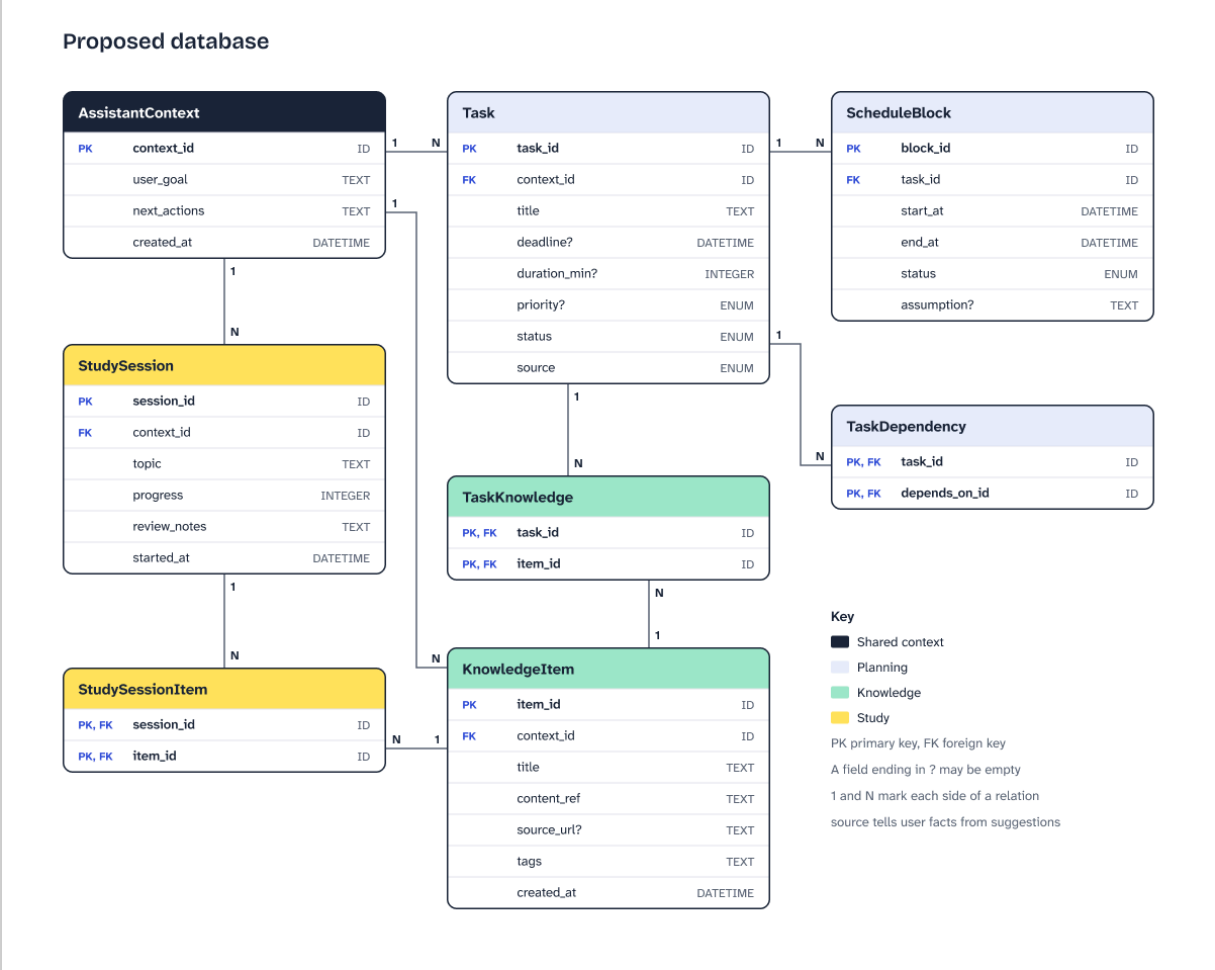


Figure 2.3. Proposed database diagram.

2.7. Computational thinking in the requirements

- **Decomposition:** Divide the problem into three features with separate inputs, processes, and outputs. Break planning into fact extraction, clarification, task management, constraint checking, scheduling, and progress tracking.
- **Pattern recognition:** Reuse workflows such as goal, action, progress check, and revision; capture, organize, retrieve, and connect; explain, practise, give feedback, and review.
- **Abstraction:** Keep details that affect study work, such as goals, tasks, deadlines, materials, and status. Ignore presentation details such as interface colour or device type when they do not affect processing. Use a shared context to connect entities without merging feature responsibilities.
- **Algorithm design:** Check for missing details before scheduling, respect stated dependencies and constraints, and report conflicts or unscheduled tasks. The team will refine and test the priority and scheduling rules with synthetic data.

Proposed scheduling workflow

1. Extract tasks and facts supplied by the student.
2. Mark required details that are missing or inconsistent. Ask the student to clarify and pause scheduling if needed.
3. Order tasks by dependency. Within each dependency level, use confirmed deadlines and priorities.
4. For each task, find an available time slot before its deadline that respects dependencies.

5. Add a suitable slot to the proposed schedule. If no slot fits, mark the task as unscheduled and record the conflicting constraint.
6. Present the schedule, assumptions, warnings, and unscheduled tasks for review.

Evaluation should check that a proposed schedule respects the supplied deadlines, durations, availability, and dependencies. Tests should also cover missing details and cases with no feasible schedule.

References

1. J. M. Wing, "Computational Thinking," *Communications of the ACM*, vol. 49, no. 3, pp. 33–35, 2006.
2. B. J. Zimmerman, "Becoming a Self-Regulated Learner: An Overview," *Theory Into Practice*, vol. 41, no. 2, pp. 64–70, 2002.
3. J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, "Improving Students' Learning With Effective Learning Techniques: Promising Directions From Cognitive and Educational Psychology," *Psychological Science in the Public Interest*, vol. 14, no. 1, pp. 4–58, 2013.